

[Logo de l'École]

[Nom de l'École / Université]

[Département / Filière]

[Logo Sagemcom]

Sagemcom Ezzahra

www.sagemcom.com

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de

[Nom du Diplôme]

Spécialité : [Spécialité]

Conception et Déploiement d'une Application de Test Telnet des Appareils Réseau avec Pipeline CI/CD DevOps

Réalisé par :

Adem Hmercha

Étudiant en [Spécialité]

[Nom de l'École / Université]

Encadrant académique :

Rim Tlili

Encadrant entreprise :

Walid Ncir

Sagemcom Ezzahra

DÉDICACE

À mes chers parents et à ma fratrie,
dont le soutien indéfectible et les encouragements
ont rendu ce parcours possible.

Votre confiance en moi a été ma plus grande force.
Dans les moments de doute, vos paroles m'ont relevé.

Votre patience lors de mes moments de stress
m'a révélé le vrai sens du mot famille.

Merci d'être ma source constante
d'inspiration et de motivation.

Avec tout mon amour et ma gratitude,

Adem Hmercha

REMERCIEMENTS

Avec une profonde gratitude, je tiens à remercier les personnes exceptionnelles qui ont rendu ce projet de fin d'études véritablement significatif, à ce moment charnière de mon parcours académique.

Tout d'abord, j'exprime ma sincère reconnaissance à **Sagemcom Ezzahra**, entreprise pionnière dans la conception d'équipements réseau embarqués et acteur de premier plan dans la transformation numérique des télécommunications. Sa confiance en m'accueillant dans son environnement dynamique a été déterminante dans l'élaboration de ce projet et dans mon développement professionnel.

Je suis particulièrement reconnaissant envers mon encadrant professionnel, **M. Walid Ncir**, dont l'expertise dans les systèmes embarqués et les infrastructures réseau a constitué ma boussole tout au long de ce parcours. Bien plus qu'un simple encadrant, il est devenu un véritable défenseur de ma réussite, m'offrant des perspectives techniques précieuses et des encouragements lors des phases les plus exigeantes du projet.

L'esprit de collaboration de l'ensemble de l'équipe du département tests de **Sagemcom Ezzahra** mérite une reconnaissance particulière. Leur volonté de partager leurs connaissances et de soutenir mon apprentissage a créé un environnement propice à l'innovation et à la progression.

Je tiens également à remercier mon encadrante académique, **Mme Rim Tlili**, dont les orientations théoriques et l'encadrement méthodologique ont garanti que ce projet réponde aux plus hautes exigences académiques tout en conservant une applicabilité concrète dans le monde professionnel.

Enfin, cette expérience n'a pas seulement abouti à un projet accompli ; elle m'a également

préparé pour le prochain chapitre de ma carrière dans le domaine en perpétuelle évolution du développement logiciel et du DevOps.

Avec sincère appréciation et respect,

Adem Hmercha

Table des matières

Introduction générale	1
1 Contexte général du projet	3
1.1 Introduction	3
1.2 Présentation de Sagemcom Ezzahra	4
1.2.1 Aperçu de l'entreprise	4
1.2.2 Position sur le marché et présence	5
1.2.3 Contexte du projet	5
1.3 Mission et objectifs du projet	5
1.3.1 Énoncé de la mission	6
1.3.2 Objectifs stratégiques	6
1.4 Périmètre du projet	7
1.4.1 Étude du système existant	7
1.4.2 Analyse critique de l'infrastructure actuelle	8
1.4.3 Solution proposée	8
1.5 Contexte et objectifs du projet	9
1.5.1 Contexte et historique	9
1.5.2 Méthodologie de travail	9
1.6 Conclusion	12

2	Analyse et spécification des besoins	14
2.1	Introduction	14
2.2	Analyse des besoins	14
2.2.1	Besoins fonctionnels	14
2.2.2	Besoins non fonctionnels	15
2.3	Conception du système	16
2.3.1	Technologies et outils clés	16
2.3.2	Architecture globale du système	19
2.4	Pipeline CI/CD	20
2.4.1	Flux de travail GitHub Actions	20
2.4.2	Configuration du self-hosted runner	20
2.4.3	Étapes du pipeline	21
2.5	Conclusion	21
3	Réalisation	22
3.1	Introduction	22
3.2	Implémentation	22
3.2.1	Structure du projet Node.js/Express	22
3.2.2	Module de test Telnet	23
3.2.3	API REST	26
3.2.4	Modèle de données MongoDB	27
3.2.5	Interface utilisateur	29
3.2.6	Conteneurisation Docker	29
3.2.7	Déploiement Kubernetes avec Helm	32
3.2.8	Pipeline GitHub Actions	34
3.2.9	Self-hosted runner	37
3.3	Conclusion	37
	Conclusion générale et perspectives	39

Bibliographie	41
----------------------	-----------

Table des figures

1.1	Logo officiel de Sagemcom	5
1.2	Schéma du processus de tests Telnet existant	7
1.3	Schéma du cadre Scrum appliqué au projet Telnet Test Manager	11
2.1	Logo Node.js / Express.js	17
2.2	Logo MongoDB	17
2.3	Logo Docker	17
2.4	Logos Kubernetes et Helm	18
2.5	Logo GitHub Actions	18
2.6	Logo Grafana	19
2.7	Architecture globale de l'application Telnet Test Manager	19
2.8	Flux de travail du pipeline GitHub Actions	20
3.1	Interface Dashboard : résultat en temps réel d'un test Telnet	26
3.2	Tableau de bord principal de l'application	29
3.3	Formulaire de lancement d'un test Telnet	29
3.4	Exécution réussie du pipeline CI/CD dans GitHub Actions	37
3.5	Runner auto-hébergé enregistré et actif	37

Liste des tableaux

1.1	Fiche d'identité de Sagemcom Ezzahra	4
1.2	Analyse critique du système de tests existant	8
1.3	Membres de l'équipe Scrum	12
2.1	Besoins fonctionnels	15
2.2	Besoins non fonctionnels	16
2.3	Composants de l'architecture et leurs rôles	19
2.4	Récapitulatif des étapes du pipeline CI/CD	21
3.1	Principaux endpoints de l'API REST	26

Introduction générale

L'industrie des équipements réseau et des télécommunications connaît une accélération technologique sans précédent. Les équipements modernes — passerelles résidentielles (*home gateways*), décodeurs multimédias (*set-top boxes*) et routeurs industriels — embarquent des systèmes d'exploitation Linux complets, accessibles à distance via le protocole Telecommunication Network Protocol (Telnet). La validation fonctionnelle de ces équipements constitue une étape critique du cycle de développement, garantissant la conformité des firmwares avant chaque mise en production.

C'est dans ce cadre que s'inscrit notre projet de fin d'études réalisé au sein de **Sagemcom Ezzahra**, centre de recherche et développement reconnu dans la conception d'équipements Customer Premises Equipment (CPE) pour les opérateurs de télécommunications internationaux. Face à l'inefficacité du processus de tests Telnet manuel en vigueur, l'équipe d'ingénierie des tests a exprimé le besoin d'une solution web centralisée, automatisée et intégrée dans un pipeline Continuous Integration / Continuous Deployment (CI/CD) moderne.

Ce rapport présente la conception et le déploiement de **Telnet Test Manager**, une application web fullstack développée selon les principes de la *Clean Architecture*, conteneurisée avec **Docker**, orchestrée par **Kubernetes**, et déployée automatiquement via un pipeline **GitHub Actions** auto-hébergé.

Le rapport est structuré en trois chapitres :

- | | |
|-------------------|---|
| Chapitre 1 | présente le contexte général du projet : l'organisme d'accueil, la problématique, les objectifs, l'analyse critique du système existant, la solution proposée et la méthodologie Scrum adoptée. |
| Chapitre 2 | détaille l'analyse et la spécification des besoins fonctionnels et non fonctionnels, les choix technologiques, l'architecture globale du système et le pipeline CI/CD. |
| Chapitre 3 | expose la réalisation complète : structure du projet, implémentation |

du moteur Telnet, API REST, base de données MongoDB, interfaces utilisateur, conteneurisation Docker, déploiement Kubernetes/Helm et configuration du pipeline GitHub Actions.

Le rapport se conclut par une synthèse des réalisations et des perspectives d'évolution de la solution.

Contexte général du projet

1.1 Introduction

Dans le paysage technologique en constante évolution de l'industrie des télécommunications, les entreprises font face à une pression croissante pour valider leurs équipements réseau avec une rapidité, une fiabilité et une rigueur sans précédent. Les approches traditionnelles de gestion des tests d'infrastructure — reposant sur des connexions Telnet manuelles, des procédures non standardisées et des résultats consignés dans des tableurs dispersés — ne sont plus suffisantes pour répondre aux exigences des environnements industriels modernes, où l'agilité opérationnelle et la capacité à livrer rapidement des firmwares validés constituent de véritables avantages concurrentiels.

Ce projet, intitulé « *Conception et Déploiement d'une Application de Test Telnet des Appareils Réseau avec Pipeline CI/CD DevOps* », répond au besoin critique d'automatisation intelligente des processus de validation au sein de **Sagemcom Ezzahra**. Alors que les équipes d'ingénierie s'efforcent d'accélérer les cycles de qualification tout en maintenant des standards élevés de fiabilité et de traçabilité, l'intégration de pratiques DevOps modernes dans les pipelines de tests émerge comme une solution transformatrice et incontournable.

La convergence des méthodologies DevOps avec l'automatisation des tests réseau représente un changement de paradigme : on passe d'opérations réactives et manuelles à une automatisation proactive et intelligente. Cette transformation permet non seulement de réduire le temps d'exécution des campagnes de validation, mais aussi d'éliminer les erreurs humaines, de centraliser la traçabilité des résultats et de créer une infrastructure de tests continue qui s'améliore à chaque livraison. Ce chapitre pose les fondements de cette transformation en présentant l'organisme d'accueil, la problématique, les objectifs stratégiques, l'analyse critique du système existant et la méthodologie de travail adoptée.

1.2 Présentation de Sagemcom Ezzahra

1.2.1 Aperçu de l'entreprise

Sagemcom est une entreprise française fondée en 2008, issue du spin-off de la division équipements de communications de Sagem. Son siège social est établi à Rueil-Malmaison (France). Sagemcom conçoit, développe et fabrique une large gamme d'équipements de communication pour les particuliers et les entreprises :

- Passerelles haut débit (CPE) xDSL, VDSL2 et fibre optique (ONT/ONU) ;
- Décodeurs multimédias (*set-top boxes*) pour la télévision numérique terrestre, câble et IPTV ;
- Compteurs communicants intelligents (Linky en France) ;
- Téléphones DECT résidentiels et professionnels.

Sagemcom Ezzahra est le centre tunisien de recherche, développement et production du groupe, situé dans la Zone Industrielle d'Ezzahra, Ben Arous, Tunisie. Ce centre emploie plus de 1 000 ingénieurs et techniciens spécialisés dans le développement logiciel embarqué, les tests de validation, l'assurance qualité et la production.

TABLE 1.1. Fiche d'identité de Sagemcom Ezzahra

Raison sociale	Sagemcom Ezzahra
Groupe	Sagemcom (France)
Fondation du groupe	2008
Secteur d'activité	Équipements de communications (CPE)
Siège Tunisie	Zone Industrielle Ezzahra, Ben Arous, Tunisie
Effectif Tunisie	> 1 000 employés
Site web	www.sagemcom.com
Activités	R&D, développement logiciel embarqué, tests, QA

1.2.2 Position sur le marché et présence

Sagemcom est **leader européen** dans la fourniture de passerelles haut débit aux opérateurs de télécommunications. Le groupe livre plus de **30 millions d'équipements** par an à travers le monde. Ses principaux clients incluent des opérateurs majeurs tels qu'Orange, SFR, Bouygues Télécom, BT, Vodafone, Deutsche Telekom et de nombreux autres opérateurs dans plus de **60 pays**.

Le centre de Sagemcom Ezzahra joue un rôle stratégique dans la compétitivité du groupe : il assure le développement et la validation des firmwares embarqués Linux pour l'ensemble de la gamme de produits CPE, garantissant la qualité et la conformité des livraisons aux opérateurs partenaires.

1.2.3 Contexte du projet



FIGURE 1.1. Logo officiel de Sagemcom

Le stage s'est déroulé au sein du **département d'ingénierie des tests**, chargé de la validation fonctionnelle des firmwares embarqués. Les équipements réseau testés (passerelles, décodeurs) embarquent un système Linux accessible à distance via le protocole Telnet sur le port 23. Chaque équipement physique (*produit*) peut exposer plusieurs interfaces Ethernet (*slots*), chacune dotée d'une adresse IP distincte, permettant des tests indépendants par interface.

1.3 Mission et objectifs du projet

1.3.1 Énoncé de la mission

Mission du projet

Développer une application web centralisée permettant d'automatiser, d'exécuter et de tracer les tests Telnet des équipements réseau embarqués de Sagemcom Ezzahra, en s'appuyant sur une infrastructure DevOps complète : conteneurisation Docker, orchestration Kubernetes et pipeline CI/CD GitHub Actions.

1.3.2 Objectifs stratégiques

Les objectifs du projet sont organisés selon quatre axes :

1. Automatisation des tests Telnet

- Éliminer la saisie manuelle des commandes Telnet ;
- Supporter les modes : commande unitaire, séquence d'étapes et supervision continue (*monitoring*) ;
- Permettre l'exécution parallèle sur plusieurs slots.

2. Centralisation et traçabilité

- Regrouper la configuration des équipements (Postes, Produits, Slots) dans une interface unique ;
- Archiver l'ensemble des résultats de tests en base de données MongoDB ;
- Fournir un journal d'audit complet des actions utilisateurs.

3. Expérience utilisateur

- Diffuser les résultats de tests en temps réel via WebSocket ;
- Implémenter un contrôle d'accès basé sur les rôles (RBAC) ;
- Proposer une interface multilingue (FR, EN, PT-BR).

4. Infrastructure DevOps

- Conteneuriser l'application avec Docker ;
- Orchestrer le déploiement via Kubernetes et Helm ;

- Automatiser l'intégration et le déploiement continu par un pipeline GitHub Actions auto-hébergé ;
- Superviser les performances avec Prometheus et Grafana.

1.4 Périmètre du projet

1.4.1 Étude du système existant

Avant la réalisation de ce projet, la procédure de validation Telnet au sein du département de tests reposait entièrement sur des opérations manuelles : chaque ingénieur ou technicien ouvrait un client Telnet (PuTTY, Windows Telnet ou un terminal Linux), saisisait l'adresse IP du slot à tester, s'authentifiait, exécutait les commandes une par une et notait visuellement les résultats dans un tableur ou un document texte.

Les principales limitations identifiées sont les suivantes :

- Saisie manuelle des commandes, source d'erreurs et de perte de temps ;
- Impossibilité d'exécuter des tests en parallèle sur plusieurs slots ;
- Aucune traçabilité structurée : résultats dispersés dans des fichiers non centralisés ;
- Dépendance totale à l'expertise individuelle de chaque technicien ;
- Absence de supervision en temps réel et de notification en cas d'échec ;
- Aucune intégration dans un processus CI/CD.

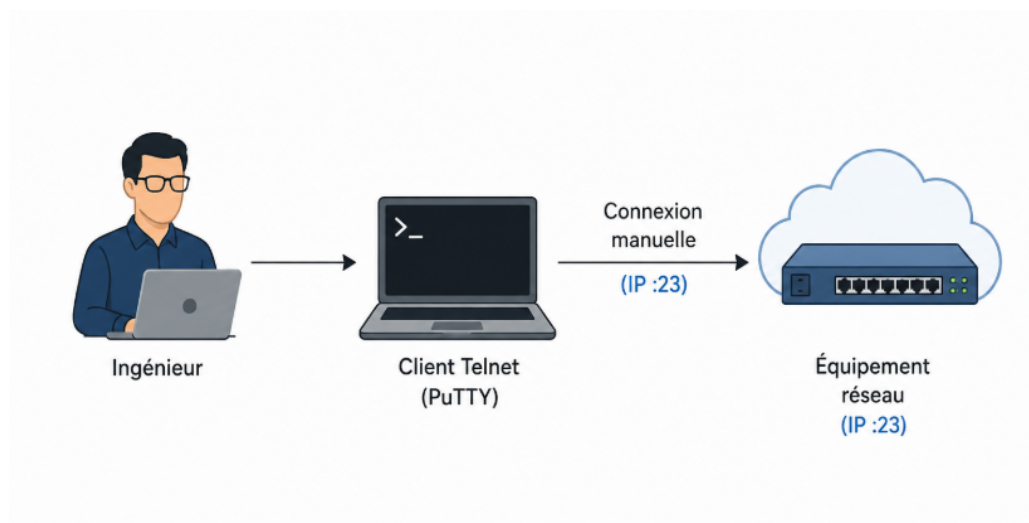


FIGURE 1.2. Schéma du processus de tests Telnet existant

1.4.2 Analyse critique de l’infrastructure actuelle

TABLE 1.2. Analyse critique du système de tests existant

Aspect	État actuel	Impact métier
Connexion Telnet	Ouverture manuelle via PuTTY ou terminal	Risque élevé d’erreurs de saisie ; temps de connexion non optimisé
Supervision	Inexistante ; contrôle visuel uniquement	Aucune détection proactive des anomalies ; pannes non tracées
Scalabilité	Un seul test à la fois, un seul slot	Allongement des cycles de validation ; blocage des livraisons
Pipeline CI/CD	Absent ; déploiement manuel	Délais de mise en production importants ; risque de régression
Qualité et standardisation	Procédures non formalisées ; dépendance aux individus	Résultats non reproductibles ; difficulté d’audit
Reporting	Fichiers Excel manuels	Aucune centralisation ; données inutilisables pour l’analyse

Analyse des causes racines :

- **Absence d’outil dédié** : aucune plateforme de tests spécifiquement conçue pour les connexions Telnet multi-slots ;
- **Manque d’industrialisation** : le processus n’est pas intégré dans une chaîne CI/CD ;
- **Défaut de centralisation** : configuration, commandes et résultats sont dispersés sans référentiel commun ;
- **Absence de gestion des utilisateurs** : aucun contrôle d’accès ni traçabilité des actions.

1.4.3 Solution proposée

La solution **Telnet Test Manager** répond à l’ensemble de ces lacunes en proposant les composants suivants :

1. Application web fullstack

- Interface React 18/TypeScript pour la configuration et l'exécution des tests ;
- Backend Node.js/Express.js structuré selon la Clean Architecture ;
- Base de données MongoDB pour la persistance.

2. Moteur de tests Telnet automatisé

- Exécution isolée dans des Worker Threads Node.js ;
- Trois modes : commande unitaire, séquence d'étapes, supervision continue ;
- Tests multi-slots en parallèle ;
- Diffusion des résultats en temps réel via WebSocket.

3. Infrastructure DevOps

- Conteneurisation Docker (build multi-stage pour le frontend) ;
- Orchestration Kubernetes avec chart Helm v0.3.0 ;
- Autoscaling horizontal (HPA : 1 à 5 répliques, seuil : 70 % CPU) ;
- Pipeline GitHub Actions en 3 jobs : CI backend, CI frontend, déploiement ;
- Supervision Prometheus/Grafana (10 panneaux de métriques).

1.5 Contexte et objectifs du projet

1.5.1 Contexte et historique

Ce projet a émergé d'un besoin concret exprimé par le département d'ingénierie des tests de Sagemcom Ezzahra. La multiplication des familles de produits (passerelles xDSL, VDSL2, fibre, décodeurs IPTV) et l'accroissement du nombre de slots à valider par équipement ont rendu le processus manuel insoutenable. Une première tentative de scripts Python avait été envisagée mais abandonnée faute de centralisation et d'interface utilisateur. Le choix s'est finalement porté sur une application web moderne, évolutive et intégrée dans l'écosystème DevOps de l'entreprise.

1.5.2 Méthodologie de travail

1.5.2.1 Mise en œuvre de la méthodologie Agile Scrum

Le projet adopte la méthodologie **Agile Scrum** [Schwaber and Sutherland, 2020] pour gérer cette initiative d'automatisation DevOps complexe. Cette approche fournit la flexibilité et les capacités de livraison itérative essentielles pour des projets de cette envergure et de cette technicité.

Avantages du cadre Scrum :

- **Livraison itérative** : permet la livraison incrémentale de chaque composant d'infrastructure (moteur Telnet, conteneurisation, orchestration, CI/CD) ;
- **Validation continue** : autorise la vérification en temps réel de chaque couche d'automatisation avant de passer à la suivante ;
- **Visibilité transparente** : fournit une visibilité claire sur l'avancement du projet à toutes les parties prenantes ;
- **Atténuation des risques** : permet l'identification et la résolution précoce des défis techniques (compatibilité Docker, configuration Kubernetes, intégration WebSocket) ;
- **Engagement des parties prenantes** : garantit un alignement continu entre les exigences techniques et les objectifs métier.

Structure des sprints :

- **Durée des sprints** : 2 semaines, optimisées pour les cycles de livraison d'infrastructure ;
- **Sprint Planning** : priorisation du backlog technique et allocation des ressources de développement ;
- **Daily Scrums** : synchronisation quotidienne de l'avancement et identification des obstacles ;
- **Sprint Reviews** : démonstration des livrables et collecte des retours des encadrants ;
- **Rétrospectives** : amélioration continue du processus et intégration des leçons apprises.

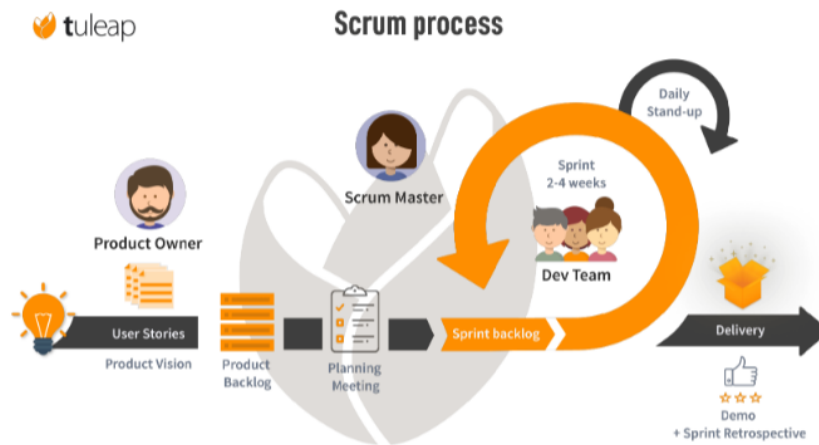


FIGURE 1.3. Schéma du cadre Scrum appliqué au projet Telnet Test Manager

Le diagramme ci-dessus illustre le flux de travail Scrum complet adapté à notre projet DevOps. Le processus englobe des cycles de sprint de deux semaines, intégrant la gestion du backlog d'infrastructure, la planification technique des sprints et les incréments livrables. Les cérémonies Daily Scrum facilitent la coordination en temps réel entre les tâches d'automatisation DevOps, l'implémentation de la sécurité et le développement des modules applicatifs.

1.5.2.2 Composition de l'équipe Scrum

L'équipe Scrum est spécifiquement structurée pour répondre à la complexité technique et aux exigences pluridisciplinaires du projet. Chaque rôle apporte une expertise spécialisée tout en maintenant un focus collaboratif sur la livraison d'une plateforme de tests prête pour la production.

TABLE 1.3. Membres de l’équipe Scrum

Rôle	Membre	Responsabilités principales
Product Owner	Walid Ncir (<i>Encadrant Sagemcom</i>)	Définition des exigences métier, priorisation du backlog produit, validation des livrables et alignement avec les objectifs de l’équipe tests de Sagemcom Ezzahra
Scrum Master	Rim Tlili (<i>Encadrante académique</i>)	Facilitation du processus Scrum, levée des obstacles, application de la méthodologie agile et suivi de l’avancement académique du projet
Équipe de développement	Adem Hmercha (<i>Étudiant PFE</i>)	Conception de l’architecture système, développement fullstack (Node.js, React, MongoDB), implémentation du moteur Telnet et déploiement de l’infrastructure DevOps complète

1.6 Conclusion

Ce chapitre a établi les fondements complets nécessaires à la mise en œuvre d’une plateforme de tests Telnet automatisée et orientée DevOps au sein de **Sagemcom Ezzahra**. L’analyse conduite révèle des opportunités de modernisation significatives, alignées à la fois sur les besoins opérationnels immédiats de l’équipe de tests et sur les objectifs stratégiques à long terme de l’entreprise.

L’évaluation de l’infrastructure existante met en évidence des lacunes manifestes en matière d’automatisation, de supervision et de traçabilité des tests, qui impactent directement l’agilité opérationnelle du département et les délais de livraison des firmwares validés. La solution proposée, **Telnet Test Manager**, répond à ces défis par une intégration systématique des pratiques DevOps modernes : moteur d’exécution Telnet automatisé, supervision temps réel par WebSocket, gestion centralisée des équipements et pipeline CI/CD de bout en bout.

La transformation depuis des opérations manuelles et réactives vers une plateforme de tests intelligente et automatisée représente bien plus qu’une simple mise à niveau technique : elle constitue un changement fondamental de philosophie opérationnelle au sein de l’équipe d’ingénierie des tests. En mettant en œuvre une automatisation complète assortie d’une supervision en temps réel, Sagemcom Ezzahra atteindra des niveaux inédits de fiabilité, de traçabilité et d’efficacité dans la validation de ses équipements réseau embarqués.

L’adoption de la méthodologie **Agile Scrum** garantit une livraison structurée et progressive des composants applicatifs et d’infrastructure complexes, tout en maintenant un focus constant

sur la qualité, la sécurité et l’optimisation des performances. La structure en cinq sprints fournit des critères de succès clairs à chaque itération et démontre la valeur opérationnelle substantielle que cette transformation est en mesure d’apporter.

Cette analyse fondatrice prépare l’équipe de développement aux phases d’implémentation technique détaillées qui suivent, en établissant des attentes précises en matière de livrables, de délais et de métriques de succès. L’intégration de la Clean Architecture, de la communication temps réel par WebSocket, de la conteneurisation Docker, de l’orchestration Kubernetes et d’un pipeline GitHub Actions constitue une plateforme robuste pour une excellence durable en matière d’automatisation des tests.

La réalisation réussie de cette plateforme de tests DevOps positionnera l’équipe d’ingénierie de Sagemcom Ezzahra comme une référence en matière d’automatisation de la validation de firmwares embarqués, permettant une exécution rapide des campagnes de tests, une traçabilité exemplaire et une productivité accrue au travers de l’application Telnet Test Manager et de ses évolutions futures.

Analyse et spécification des besoins

2.1 Introduction

Ce chapitre présente l'analyse complète des besoins fonctionnels et non fonctionnels de l'application Telnet Test Manager, ainsi que les choix technologiques et architecturaux justifiés par ces besoins. Nous détaillons également la conception du pipeline CI/CD qui automatise l'intégration et le déploiement continus de la solution.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

Les besoins fonctionnels définissent les capacités essentielles que l'application Telnet Test Manager doit fournir pour répondre aux exigences opérationnelles du département tests de Sagemcom Ezzahra.

TABLE 2.1. Besoins fonctionnels

Test de connectivité Telnet	Connexion TCP automatique sur le port 23 d'un slot, authentification, exécution de commande et validation de la réponse obtenue.
Gestion des équipements	Création, modification et suppression des Postes, Produits, Slots (IP : port) et Références via une interface CRUD complète.
Bibliothèque de commandes	Gestion de commandes Telnet réutilisables de types : commande unitaire, séquence d'étapes et supervision continue (<i>monitoring</i>).
Tests multi-slots en parallèle	Lancement simultané de tests Telnet sur plusieurs interfaces Ethernet d'un ou plusieurs équipements via des Worker Threads isolés.
Résultats en temps réel	Diffusion des événements de test (logs, progression, statut de chaque étape) via WebSocket avec abonnement par identifiant de test.
Authentification et contrôle d'accès	Connexion sécurisée par JWT, système RBAC avec trois rôles (Administrateur, Ingénieur, Technicien) et limitation de débit sur les tentatives de connexion.
Tableau de bord d'administration	Gestion des utilisateurs, consultation des journaux d'audit, statistiques d'utilisation et analytiques des campagnes de tests.
Génération de rapports	Compilation de résultats de tests en rapports synthétiques, avec historique complet filtrable par statut, date et slot.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels garantissent que le système répond aux exigences de qualité industrielle en termes de performance, sécurité, disponibilité et maintenabilité.

TABLE 2.2. Besoins non fonctionnels

Catégorie	Exigence	Critère de mesure
Performance	Temps de réponse API	95 % des requêtes non-Telnet en moins de 2 secondes
	Parallélisme	10 tests Telnet simultanés minimum
Sécurité	Authentification	JWT avec expiration configurable, chiffrement HS256
	Mots de passe	Hachage bcryptjs, facteur de coût : 10
	En-têtes HTTP	Helmet.js : CSP, HSTS, X-Frame-Options
Disponibilité	Objectif de service	99,9 % sur l’infrastructure locale
Scalabilité	Autoscaling	HPA Kubernetes : 1 à 5 répliques, seuil CPU 70 %
	Sans état	Backend stateless, session portée par le JWT
Maintenabilité	Architecture	Clean Architecture 4 couches, principes SOLID
Portabilité	Conteneurisation	Images Docker, déploiement identique dev/production

2.3 Conception du système

2.3.1 Technologies et outils clés

2.3.1.1 Node.js et Express.js

Node.js (version 20 LTS) est un environnement d’exécution JavaScript côté serveur basé sur le moteur V8 de Google. Son modèle non bloquant (*event loop*) et la prise en charge native des *Worker Threads* en font un choix idéal pour un serveur Telnet concurrent. **Express.js** (v4.18) est le framework HTTP minimaliste le plus utilisé dans l’écosystème Node.js ; il structure l’application en routes, middlewares et contrôleurs.

**FIGURE 2.1.** Logo Node.js / Express.js

2.3.1.2 MongoDB

MongoDB 7 est un système de gestion de base de données orienté documents (Not Only SQL (NoSQL)). Les données sont stockées en Binary JSON (BSON), un format binaire dérivé du JavaScript Object Notation (JSON). Cette flexibilité schématique est particulièrement adaptée aux structures de données hétérogènes des résultats de tests Telnet. L'Object-Relational Mapping (ORM) **Mongoose 9** fournit la validation des schémas et les middlewares d'accès aux données.

**FIGURE 2.2.** Logo MongoDB

2.3.1.3 Docker et Docker Compose

Docker est une plateforme de conteneurisation qui encapsule une application et ses dépendances dans des images légères et portables. **Docker Compose** orchestre plusieurs conteneurs localement via un fichier YAML Ain't Markup Language (YAML) déclaratif. La stack de développement comprend cinq services : MongoDB, backend, frontend, Prometheus et Grafana.

**FIGURE 2.3.** Logo Docker

2.3.1.4 Kubernetes et Helm

Kubernetes (Kubernetes (K8s)) est le système d'orchestration de conteneurs open source de référence. Il automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. L'environnement utilisé est **Minikube** (cluster local). **Helm** est le gestionnaire de paquets de Kubernetes ; il paramétrise les manifestes YAML via des templates et des fichiers de valeurs, facilitant la reproductibilité des déploiements.



FIGURE 2.4. Logos Kubernetes et Helm

2.3.1.5 GitHub Actions et self-hosted runner

GitHub Actions est la plateforme d'CI/CD intégrée à GitHub. Les workflows sont définis en YAML et déclenchés par des événements Git (push, pull request). Un **runner auto-hébergé** (*self-hosted runner*) est un agent installé sur la machine locale hébergeant Minikube, permettant d'accéder au cluster Kubernetes sans l'exposer sur Internet.



FIGURE 2.5. Logo GitHub Actions

2.3.1.6 Prometheus et Grafana

Prometheus est un système de collecte de métriques open source qui scrape périodiquement les endpoints `/metrics` exposés par les applications. **Grafana** est la plateforme de visualisation associée qui permet de construire des tableaux de bord interactifs à partir des métriques collectées. Le projet expose quatre métriques personnalisées : requêtes HTTP, durées de réponse, tests Telnet lancés et connexions WebSocket actives.



FIGURE 2.6. Logo Grafana

2.3.2 Architecture globale du système

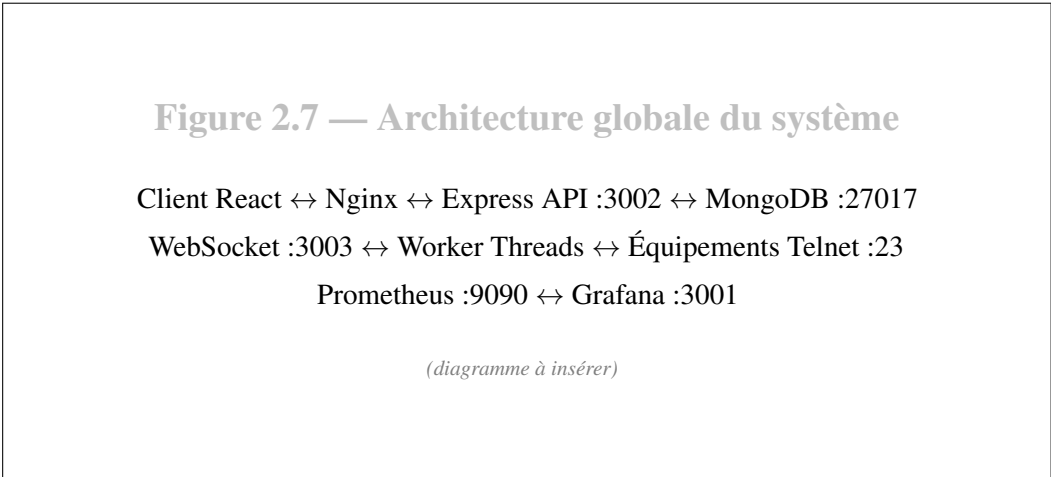


FIGURE 2.7. Architecture globale de l’application Telnets Test Manager

TABLE 2.3. Composants de l’architecture et leurs rôles

Composant	Technologie	Rôle
Interface client	React 18 + TS	SPA : configuration, exécution, supervision
Proxy inverse	Nginx	Routage des requêtes HTTP et WebSocket
API REST	Express.js	Logique métier, 32 endpoints sécurisés
Serveur WebSocket	ws 8.19	Diffusion temps réel des événements de test
Moteur Telnets	Worker Threads	Exécution isolée des sessions Telnets
Base de données	MongoDB 7	Persistance : utilisateurs, équipements, tests
Métriques	Prometheus	Collecte de 4 métriques applicatives
Tableaux de bord	Grafana	Visualisation des performances en temps réel

2.4 Pipeline CI/CD

2.4.1 Flux de travail GitHub Actions

Le pipeline GitHub Actions est déclenché automatiquement à chaque poussée sur les branches `main` et `develop`, ainsi qu'à l'ouverture d'une *pull request* vers `main`. Un mécanisme de concurrence annule les exécutions précédentes pour la même branche, évitant les déploiements redondants.

Figure 2.8 — Flux du pipeline CI/CD

git push → GitHub → CI Backend + CI Frontend (parallèle) → Deploy (self-hosted)

(diagramme à insérer)

FIGURE 2.8. Flux de travail du pipeline GitHub Actions

2.4.2 Configuration du self-hosted runner

Le **runner auto-hébergé** est installé sur la machine Windows locale hébergeant Minikube. Il est enregistré auprès du dépôt GitHub avec les labels `self-hosted`, `windows` et `minikube`. Ce runner est le seul autorisé à exécuter le job `deploy`, qui requiert un accès direct au cluster Kubernetes local et au daemon Docker de Minikube.

2.4.3 Étapes du pipeline

TABLE 2.4. Récapitulatif des étapes du pipeline CI/CD

Étape	Action	Outil	Condition d'échec
Audit sécurité	<code>npm audit</code> <code>--audit-level=critical</code>	npm	Vulnérabilité critique détectée
Tests	Tests unitaires et d'intégration	Jest, Supertest	Échec d'au moins un test
Build frontend	<code>npm run build</code> (React → statiques)	React Scripts	Erreur de compilation TypeScript
Build Docker	Construction des images backend et frontend	Docker	Erreur dans un Dockerfile
Deploy Helm	<code>helm upgrade</code> <code>--install --atomic</code>	Helm, kubectl	Pod en erreur après timeout 600s
Validation	<code>kubectl rollout status</code>	kubectl	Timeout de disponibilité

2.5 Conclusion

Ce chapitre a formalisé l'ensemble des besoins fonctionnels et non fonctionnels de Telnet Test Manager sous une forme structurée et lisible. Les huit besoins fonctionnels couvrent l'intégralité du cycle de vie des tests Telnet, de la connexion initiale jusqu'à la génération des rapports. Les exigences non fonctionnelles garantissent une qualité industrielle en termes de performance, sécurité, scalabilité et maintenabilité.

Les six technologies retenues — Node.js, MongoDB, Docker, Kubernetes, GitHub Actions et Prometheus/Grafana — forment un écosystème cohérent et éprouvé, parfaitement adapté aux contraintes d'un déploiement DevOps sur infrastructure locale. Le pipeline CI/CD en six étapes automatise l'ensemble du cycle de livraison avec des contrôles de qualité à chaque étape. Le chapitre suivant détaille la réalisation technique de l'ensemble de ces composants.

Réalisation

3.1 Introduction

Ce chapitre présente la réalisation concrète de l'application Telnet Test Manager. Nous parcourons successivement la structure du projet backend, l'implémentation du moteur Telnet, l'API REST, le modèle de données MongoDB, les interfaces utilisateur, la conteneurisation Docker, le déploiement Kubernetes/Helm, le pipeline GitHub Actions et la configuration du runner auto-hébergé.

3.2 Implémentation

3.2.1 Structure du projet Node.js/Express

Le backend adopte la **Clean Architecture** en quatre couches dont les dépendances ne pointent que vers l'intérieur (Domain $\leftarrow$ Application $\leftarrow$ Infrastructure $\leftarrow$ Interfaces).

```
1 backend /
2     src /
3         domain /                                # Entités et
4         interfaces (aucune dépendance externe)
5             entities /                            # User , Slot ,
6             TelnetCommand , TestResult ...
7             repositories /                        # IUserRepository ,
            ISlotRepository ...
            application /
                usecases /                          # LoginUseCase ,
            RunTestUseCase , GetResultsUseCase ...
```

```

8         infrastructure/
9             config/database.js      # Connexion MongoDB
10            database/
11                models/              # 9 schémas
Mongoose
12                repositories/       # 9
implémentations MongoDB
13                telnet/
14                    TestWorkerManager.js # Gestion du
cycle de vie des Workers
15                    testWorker.js     # Logique Telnet
isolée dans un Worker Thread
16                metrics/
17                    prometheusMetrics.js # 4 métriques
Prometheus
18                interfaces/
19                    http/
20                        controllers/   # 9 contrôleurs
minces
21                        routes/        # 10 groupes de
routes Express
22                        middlewares/   # authenticate,
requireRole, auditLog, errorHandler
23                        websocket/
24                            WebSocketServer.js # Serveur WS port
3003
25                main/
26                    server.js          # Point d'entrée
27                    app.js             # Configuration
Express
28                container.js           # Conteneur
d'injection de dépendances
29                Dockerfile
30                package.json

```

Listing 3.1. Arborescence du projet backend

3.2.2 Module de test Telnet

Le cœur du système est le moteur Telnet, composé du **TestWorkerManager** (gestionnaire du cycle de vie) et du **testWorker** (logique d'exécution dans un Worker Thread isolé).

```
1  const { workerData, parentPort } = require('worker_threads');
2  const Telnet = require('telnet-client');
3
4  async function runTest() {
5      const { slot, command, testId } = workerData;
6      const telnet = new Telnet();
7
8      // Paramètres de connexion à l'équipement réseau
9      const params = {
10         host:                slot.adresse,    // ex : 192.168.5.1
11         port:                slot.port,       // 23 (Telnet)
12         timeout:             15000,
13         shellPrompt:         /\s*$/,          // Invite shell Linux
14                                (root)
15         loginPrompt:         /login:\s*$/,
16         passwordPrompt:      /Password:\s*$/,
17         username:            'root',
18         password:            'root',
19         negotiationMandatory: true,
20         stripControls:       true,
21         echolines:           0,
22     };
23
24     try {
25         await telnet.connect(params);
26
27         if (command.type === 'single') {
28             // Mode commande unitaire
29
30             const response = await telnet.exec(command.command);
31             const success = !command.expectedResponse ||
32                             response.includes(command.expectedResponse);
33             parentPort.postMessage({ type: 'completed', testId,
34                                     success, response });
35
36         } else if (command.type === 'sequence') {
37             // Mode séquence : étapes ordonnées
38
39             for (let i = 0; i < command.steps.length; i++) {
40                 const step = command.steps[i];
41                 const res = await telnet.exec(step.command);
```

```
38     const ok    = !step.expectedResponse ||
39                 res.includes(step.expectedResponse);
40     // Envoyer le résultat de l'étape au thread principal
41     parentPort.postMessage({
42         type: 'step', testId,
43         stepIndex: i,
44         status:    ok ? 'SUCCESS' : 'FAIL',
45         response:  res,
46         timestamp: new Date().toISOString(),
47     });
48 }
49 parentPort.postMessage({ type: 'completed', testId,
50                             success: true });
51
52 } else if (command.type === 'monitoring') {
53     // Mode supervision : écoute d'événements
54
55     await telnet.exec(command.command);
56     const duration = command.steps[0]?.duration || 15000;
57     await new Promise(resolve => setTimeout(resolve,
58         duration));
59     parentPort.postMessage({ type: 'completed', testId,
60                             success: true });
61 }
62
63 await telnet.end();
64
65 } catch (err) {
66     // Signaler l'erreur au thread principal
67     parentPort.postMessage({ type: 'error', testId, message:
68         err.message });
69 }
70
71 }
```

Listing 3.2. testWorker.js — exécution d'un test Telnet en mode séquence

Figure 3.1 — Résultat d’un test Telnet en temps réel

Capture de l’interface Dashboard : terminal de résultats avec statuts
SUCCESS/FAIL par étape

(capture d’écran à insérer)

FIGURE 3.1. Interface Dashboard : résultat en temps réel d’un test Telnet

3.2.3 API REST

TABLE 3.1. Principaux endpoints de l’API REST

Méthode	Endpoint	Description	Auth
POST	/login	Authentification JWT	Non
POST	/logout	Déconnexion	JWT
GET	/postes	Liste des postes	JWT
POST	/postes	Créer un poste	Ingénieur
GET	/produits	Liste des produits	JWT
GET	/slots	Liste des slots	JWT
POST	/slots	Créer un slot	Ingénieur
GET	/telnet-commands	Bibliothèque commandes	JWT
POST	/telnet-commands	Créer une commande	Ingénieur
POST	/run-test	Lancer un test Telnet	JWT
POST	/stop-test	Arrêter un test	JWT
POST	/stop-monitoring	Arrêter la supervision	JWT

Table 3.1 — suite			
Méthode	Endpoint	Description	Auth
GET	/test-results	Historique des résultats	JWT
GET	/reports	Liste des rapports	JWT
POST	/reports/generate	Générer un rapport	JWT
GET	/admin/users	Gestion utilisateurs	Admin
POST	/admin/users	Créer un utilisateur	Admin
GET	/admin/audit-logs	Journal d’audit	Admin
GET	/admin/analytics	Statistiques globales	Admin
GET	/metrics	Métriques Prometheus	Public
GET	/health	Statut du serveur	Public

3.2.4 Modèle de données MongoDB

```

1  const mongoose = require('mongoose');
2
3  const SlotSchema = new mongoose.Schema({
4    id:          { type: Number, required: true, unique: true },
5    nom:         { type: String, required: true },
6    produitId:   { type: Number, required: true, ref: 'Produit' },
7    adresse:     { type: String, required: true }, // Adresse IP
               : ex. "192.168.5.1"
8    port:       { type: Number, default: 23 },    // Port Telnet
               standard
9    description: { type: String, default: '' },
10 });
11
12 module.exports = mongoose.model('Slot', SlotSchema);

```

Listing 3.3. Schéma Mongoose — collection slots

```
1 const TestResultSchema = new mongoose.Schema({
2   id:          { type: Number, unique: true },    // Timestamp :
               Date.now()
3   slotId:      { type: Number, ref: 'Slot' },
4   posteId:     { type: Number, ref: 'Poste' },
5   produitId:   { type: Number, ref: 'Produit' },
6   commandId:   { type: String, ref: 'TelnetCommand' },
7   runMode:     {
8     type: String,
9     enum: ['single', 'builtin_sequence', 'sequence',
10            'monitoring']
11  },
12  statut: {
13    type: String,
14    enum: ['PENDING', 'SUCCESS', 'FAIL', 'STOPPED'],
15    default: 'PENDING'
16  },
17  startTime: { type: Date },
18  endTime:   { type: Date },
19  steps: [{
20    step:      String,
21    description: String,
22    statut:     { type: String, enum: ['SUCCESS', 'FAIL'] },
23    timestamp:  Date,
24  }],
25  logs: [{ type: String }],
26 });
```

Listing 3.4. Schéma Mongoose — collection testResults

3.2.5 Interface utilisateur

Figure 3.2 — Tableau de bord principal (Dashboard)

Panneau de sélection (Poste / Produit / Slot / Commande) + terminal de résultats en temps réel via WebSocket

(capture d'écran à insérer)

FIGURE 3.2. Tableau de bord principal de l'application

Figure 3.3 — Formulaire de lancement d'un test Telnet

Sélection du slot, choix de la commande, options d'exécution et bouton « Lancer le test »

(capture d'écran à insérer)

FIGURE 3.3. Formulaire de lancement d'un test Telnet

3.2.6 Conteneurisation Docker

```
1 # Image de base légère : Node.js 20 sur Alpine Linux (~50 MB)
2 FROM node:20-alpine
3
4 WORKDIR /app
5
6 # Copie des manifestes pour exploiter le cache Docker
7 COPY package*.json ./
8
9 # Installation des dépendances de production uniquement
10 RUN npm ci --only=production
11
12 # Cache-busting : force la recopie du code source à chaque build
13 ARG CACHEBUST=1
14 COPY . .
15
```



```

16 EXPOSE 3002
17
18 CMD ["node", "src/main/server.js"]

```

Listing 3.5. backend/Dockerfile — image Node.js 20 Alpine

```

1  #           tape 1 : Construction de l'application React
2  FROM node:18-alpine AS builder
3  WORKDIR /app
4  COPY package*.json ./
5  RUN npm ci
6  COPY . .
7  # Génère les fichiers statiques optimisés dans ./build
8  RUN npm run build
9
10 #           tape 2 : Serveur Nginx pour les fichiers statiques
11 FROM nginx:alpine
12 # L'image finale ne contient que Nginx + les fichiers compilés
   (~25 MB)
13 COPY --from=builder /app/build /usr/share/nginx/html
14 COPY nginx.conf /etc/nginx/conf.d/default.conf
15 EXPOSE 80
16 CMD ["nginx", "-g", "daemon off;"]

```

Listing 3.6. frontend/Dockerfile — build multi-stage React + Nginx

```

1  services:
2    mongodb:
3      image: mongo:7
4      ports:
5        - "27017:27017"
6      environment:
7        MONGO_INITDB_ROOT_USERNAME: admin
8        MONGO_INITDB_ROOT_PASSWORD: <MOT_DE_PASSE>  # à définir
   dans .env
9        MONGO_INITDB_DATABASE:      telnet-db
10     volumes:
11       - mongo-data:/data/db
12

```

```

13  backend:
14      build: ./backend
15      ports:
16          - "3002:3002"      # API REST
17          - "3003:3003"      # WebSocket
18      environment:
19          MONGO_URI:
20              mongodb://admin:<MOT_DE_PASSE>@mongodb:27017/telnet-db?authSource=
21          JWT_SECRET:      ${JWT_SECRET}      # injecté depuis le
22              fichier .env
23          ALLOWED_ORIGIN: http://localhost:3000
24      depends_on: [mongodb]
25
26  frontend:
27      build: ./frontend
28      ports:
29          - "3000:80"
30      depends_on: [backend]
31
32  prometheus:
33      image: prom/prometheus:latest
34      ports:
35          - "9090:9090"
36      volumes:
37          -
38              ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
39
40  grafana:
41      image: grafana/grafana:latest
42      ports:
43          - "3001:3000"
44      environment:
45          GF_SECURITY_ADMIN_PASSWORD: <MOT_DE_PASSE>      # à définir
46              dans .env
47      volumes:
48          -
49              ./monitoring/grafana-datasources.yml:/etc/grafana/provisioning/dash
50          -
51              ./monitoring/grafana-dashboard.json:/etc/grafana/provisioning/dash
52      depends_on: [prometheus]

```

```

48 volumes :
49     mongo-data :

```

Listing 3.7. docker-compose.yml — stack locale de développement (5 services)

3.2.7 Déploiement Kubernetes avec Helm

```

1 helm/telnet-app/
2     Chart.yaml                                # Métadonnées (name,
   version: 0.3.0)
3     values.yaml                              # 40+ valeurs
   paramétrables
4     templates/
5         backend-deployment.yaml              # Deployment backend (2
   répliques)
6         backend-hpa.yaml                    # HPA : 1-5 répliques,
   seuil CPU 70 %
7         backend-pvc.yaml                    # PVC 1 Gi pour
   /app/reports
8         backend-service.yaml                # ClusterIP : 3002
9         frontend-deployment.yaml            # Deployment frontend
   (1 réplique)
10        frontend-service.yaml               # ClusterIP : 80
11        mongodb-deployment.yaml             # Deployment MongoDB 7
12        mongodb-pvc.yaml                   # PVC 2 Gi pour /data/db
13        mongodb-service.yaml               # ClusterIP : 27017
14        configmap.yaml                     # Variables
   d'environnement
15        secret.yaml                        # JWT_SECRET,
   credentials MongoDB
16        ingress.yaml                       # Nginx ingress : /
   frontend, /api      backend
17        seed-job.yaml                      # Hook post-install :
   seed admin + engineer

```

Listing 3.8. Structure du chart Helm telnet-app (v0.3.0)

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:

```

```

4   name: {{ .Release.Name }}-backend
5   namespace: {{ .Release.Namespace }}
6   spec:
7     replicas: {{ .Values.replicaCount.backend }}
8     strategy:
9       type: RollingUpdate
10    rollingUpdate:
11      maxUnavailable: 0    # Aucun pod indisponible pendant la
                           mise à jour
12      maxSurge:           1    # Un pod supplémentaire temporaire
                           autorisé
13  selector:
14    matchLabels:
15      app: {{ .Release.Name }}-backend
16  template:
17    spec:
18      # Attendre que MongoDB soit prêt avant de démarrer le
                           backend
19    initContainers:
20      - name: wait-for-mongodb
21        image: busybox:1.35
22        command: ['sh', '-c',
23          'until nc -z {{ .Release.Name }}-mongodb 27017;
24            do echo waiting; sleep 2; done']
25    containers:
26      - name: backend
27        image: "telnet-backend:{{ .Values.image.backend.tag
28          }}"
29        ports:
30          - containerPort: 3002    # API REST
31          - containerPort: 3003    # WebSocket
32        resources:
33          requests:
34            memory: {{
35              .Values.resources.backend.requests.memory }}
36            cpu:     {{ .Values.resources.backend.requests.cpu
37              }}
38          limits:
39            memory: {{
40              .Values.resources.backend.limits.memory }}
41            cpu:     {{ .Values.resources.backend.limits.cpu }}

```

Listing 3.9. Extrait — backend-deployment.yaml (chart Helm)

```

1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: {{ .Release.Name }}-backend-hpa
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind:      Deployment
9      name:      {{ .Release.Name }}-backend
10   minReplicas: {{ .Values.hpa.backend.minReplicas }} # 1
11   maxReplicas: {{ .Values.hpa.backend.maxReplicas }} # 5
12   metrics:
13     - type: Resource
14       resource:
15         name: cpu
16         target:
17           type:      AverageUtilization
18           averageUtilization: {{
19             .Values.hpa.backend.targetCPUUtilizationPercentage
20           }} # 70

```

Listing 3.10. backend-hpa.yaml — Horizontal Pod Autoscaler

3.2.8 Pipeline GitHub Actions

```

1  name: "CI/CD      Telnet Test Manager"
2
3  on:
4    push:
5      branches: [main, develop]
6    pull_request:
7      branches: [main]
8
9  # Annuler les exécutions précédentes pour la même branche
10 concurrency:
11   group: deploy-{{ github.ref }}
12   cancel-in-progress: true

```

```
13
14 jobs:
15   #
16
17   # JOB 1      Intégration continue Backend
18   #
19   ci-backend:
20     runs-on: ubuntu-latest
21     steps:
22       - uses: actions/checkout@v4
23       - uses: actions/setup-node@v4
24         with: { node-version: '20' }
25       - run: cd backend && npm ci
26       - run: cd backend && npm audit --audit-level=critical
27       - run: cd backend && npm test --if-present
28   #
29
30   # JOB 2      Intégration continue Frontend
31   #
32   ci-frontend:
33     runs-on: ubuntu-latest
34     steps:
35       - uses: actions/checkout@v4
36       - uses: actions/setup-node@v4
37         with: { node-version: '20' }
38       - run: cd frontend && npm ci
39       - run: cd frontend && npm audit --audit-level=critical
40       - run: cd frontend && npm run build
41       env: { CI: false }
42   #
43
44   # JOB 3      Déploiement (self-hosted, branche main uniquement)
45   #
46   deploy:
47     runs-on: self-hosted
48     needs: [ci-backend, ci-frontend]
```

```

48   if: github.ref == 'refs/heads/main'
49   steps:
50     - uses: actions/checkout@v4
51
52     - name: "Démarrer Minikube si nécessaire"
53       run: minikube status || minikube start --driver=docker
54
55     - name: "Construire les images Docker (daemon Minikube)"
56       run: |
57         eval $(minikube docker-env)
58         docker build -t telnet-backend:latest ./backend
59         docker build -t telnet-frontend:latest ./frontend
60
61     - name: "Déploiement Helm (atomic)"
62       run: |
63         helm upgrade --install telnet-app ./helm/telnet-app \
64           --create-namespace --namespace telnet-app \
65           --set secrets.mongoRootPassword=${{
66             secrets.MONGO_PASSWORD }} \
67           --set secrets.jwtSecret=${{ secrets.JWT_SECRET }} \
68           \
69           --atomic --timeout 600s
70
71     - name: "Attendre la disponibilité des pods"
72       run: |
73         kubectl rollout status deployment/telnet-app-backend
74         \
75         -n telnet-app --timeout=240s
76         kubectl rollout status deployment/telnet-app-frontend
77         \
78         -n telnet-app --timeout=180s
79
80     - name: "Résumé du déploiement"
81       run: kubectl get pods,services -n telnet-app

```

Listing 3.11. `.github/workflows/ci-cd.yaml` — pipeline CI/CD complet

Figure 3.4 — Exécution du pipeline dans GitHub Actions

Vue de l'onglet Actions : 3 jobs (CI Backend, CI Frontend, Deploy) avec statuts
Success/Failure

(capture d'écran à insérer)

FIGURE 3.4. Exécution réussie du pipeline CI/CD dans GitHub Actions

3.2.9 Self-hosted runner

Le runner auto-hébergé est installé sur la machine Windows locale. L'enregistrement auprès du dépôt GitHub s'effectue via :

```
1 # Depuis le dossier d'installation du runner GitHub Actions
2 .\config.cmd --url https://github.com/[ORGANISATION]/[REPO] '
3               --token [TOKEN_RUNNER]                      '
4               --name "minikube-runner-local"              '
5               --labels self-hosted,windows,minikube
6
7 # Démarrage du runner en écoute permanente
8 .\run.cmd
```

Listing 3.12. Commandes d'enregistrement du runner auto-hébergé

Figure 3.5 — Runner auto-hébergé actif sur GitHub

Paramètres du dépôt → Actions → Runners : statut « Idle » (en attente de jobs)

(capture d'écran à insérer)

FIGURE 3.5. Runner auto-hébergé enregistré et actif

3.3 Conclusion

Ce chapitre a détaillé la réalisation complète de Telnet Test Manager, depuis l'implémentation du moteur Telnet en Worker Threads jusqu'au pipeline de déploiement GitHub Actions.

La conteneurisation Docker multi-stage, l'orchestration Kubernetes/Helm avec autoscaling horizontal et la surveillance Prometheus/Grafana constituent une infrastructure DevOps de niveau production. L'ensemble des fonctionnalités réalisées répondent aux seize besoins fonctionnels identifiés au chapitre précédent.

Conclusion générale et perspectives

Bilan des réalisations

Ce projet de fin d'études, réalisé au sein de **Sagemcom Ezzahra**, avait pour ambition de concevoir et déployer une solution complète de tests Telnet automatisés pour les équipements réseau embarqués, intégrée dans une infrastructure DevOps moderne.

Sur le **plan applicatif**, nous avons livré **Telnet Test Manager**, une application web fullstack structurée selon les principes de la *Clean Architecture*. Le moteur Telnet, implémenté via les Worker Threads de Node.js et la bibliothèque `telnet-client`, supporte quatre modes d'exécution : commande unitaire, séquence, supervision continue et séquence personnalisée. La diffusion des résultats en temps réel par WebSocket et l'interface React 18/TypeScript multilingue offrent une expérience utilisateur professionnelle et réactive.

Sur le **plan DevOps**, l'application est entièrement conteneurisée avec Docker (build multi-stage pour le frontend, réduisant l'image de production à ~25 Mo), orchestrée par un chart Helm v0.3.0 sur Kubernetes Minikube avec autoscaling horizontal (1 à 5 répliques, seuil CPU : 70 %). Un pipeline GitHub Actions en trois jobs automatise l'intégration continue, les audits de sécurité et le déploiement via un runner auto-hébergé. La stack Prometheus/Grafana assure la visibilité en temps réel des performances applicatives.

Perspectives d'évolution

Plusieurs axes d'amélioration sont envisagés pour les versions futures :

- **Support du protocole SSH** : ajouter une couche SSH en complément de Telnet pour couvrir les équipements modernes refusant les connexions non chiffrées ;
- **Tests end-to-end automatisés** : intégrer Cypress ou Playwright dans le pipeline CI/CD pour valider automatiquement les scénarios utilisateur complets ;

- **Analyse intelligente des résultats** : appliquer des algorithmes de détection d'anomalies sur l'historique des tests pour anticiper les défaillances matérielles ;
- **Déploiement cloud hybride** : migrer vers AWS EKS ou GCP GKE pour étendre la portée de l'infrastructure à plusieurs sites géographiques ;
- **Planification des campagnes** : implémenter un module de scheduling permettant de lancer des campagnes de tests automatiques sur des plages horaires programmées.

En définitive, ce projet illustre comment l'union du génie logiciel moderne (*Clean Architecture*, WebSocket, Role-Based Access Control (RBAC)) et des pratiques Development and Operations (DevOps) avancées (Docker, Kubernetes, CI/CD, Prometheus) peut transformer un processus manuel fragmenté en une plateforme automatisée, traçable et scalable, apportant une valeur ajoutée concrète à l'équipe d'ingénierie de Sagemcom Ezzahra.

Bibliographie

[Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide.
Consulté en décembre 2024.